

Semester Project: Domestic Airline Flight Database

Mission Statement:

The scope of the project is to design a database system capable of capturing up-to-the-minute operational information for a prospective airline, Domestic Airlines. The system will collect and manage comprehensive data related to flights, airplanes, passengers, pricing, and other relevant operational entities. This project is also motivated by the desire to challenge ourselves with the complexity inherent in real-world airport databases, which must manage vast quantities of information and constantly shifting data flows. Because such systems require strong regulation, organization, and reliable data storage, our model aims to present a streamlined yet representative view of the types of data an airline would need to maintain, as well as how that data should be structured within a database environment. This information will serve a dual purpose. First, it will support internal functions by providing Domestic Airlines and its associated partners with accurate, timely, and well-structured data to inform decision-making and enhance operational efficiency. Second, the collected data will be organized and maintained in a format suitable for reporting to regulatory agencies, including governing bodies such as the Federal Aviation Administration (FAA), to ensure compliance with industry standards and aviation regulations.

Conceptual design:

Our database model is composed of seven core tables: Airport, Airplane, Flight, Booking, Passenger, FrequentFlyer, and Payment. The structure of these tables and the selection of their attributes are informed by Federal Aviation Administration (FAA) documentation, supplemented by our own practical observations and experiences related to airport operations.

Entity Rules:

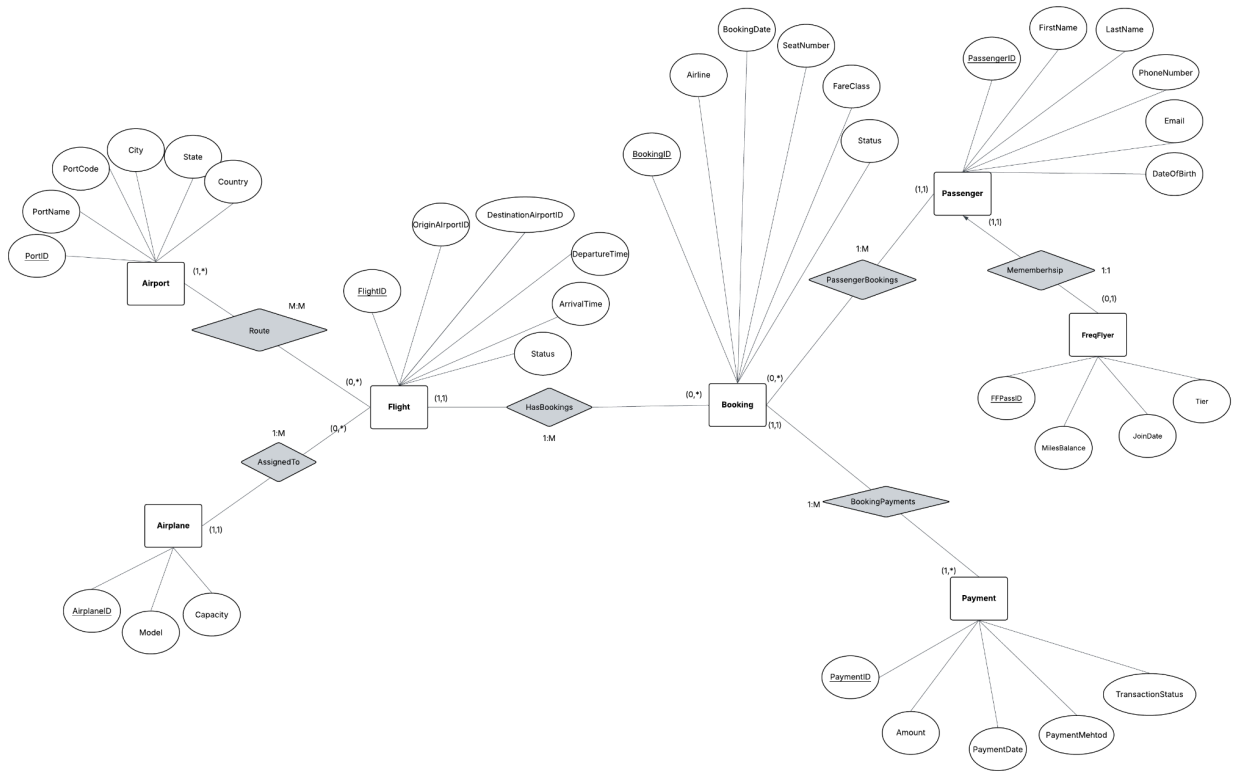
1. Airport
 - a. Each airport is uniquely identified by PortID
 - b. For every airport, the system stores its PortName, PortCodemCity, State
2. Airplane
 - a. Each airplane is uniquely identified by AirplaneID
 - b. For every airplane, the system stores its Model and capacity (the number of seats on the plane)
3. Flight
 - a. Each flight is uniquely identified by BookinID
 - b. A booking stores the Airline, BookingDate, SeatNumber, FareClass and Statues
4. Booking
 - a. Each booking is uniquely identified by BookingID

- b. A booking stores the Airline, BookingDate, SearNumber, FareClass, and Statues
- 5. Passenger
 - a. Each passenger is uniquely identified by PassengerID
 - b. For every passenger the system stores FirstName, LastName, PhoneNumber, Email; and DateOfBirth
- 6. FrequentFlyer
 - a. Each frequent-flyer account is uniquely identified FFPassID
 - b. A frequent-flyer record stores MilesBalance, JoinDate, and Tier
- 7. Payment
 - a. Each payment is uniquely identified by PaymentID
 - b. A payment records the Amount, PaymentDate, PaymentMethod and TransactionStatus

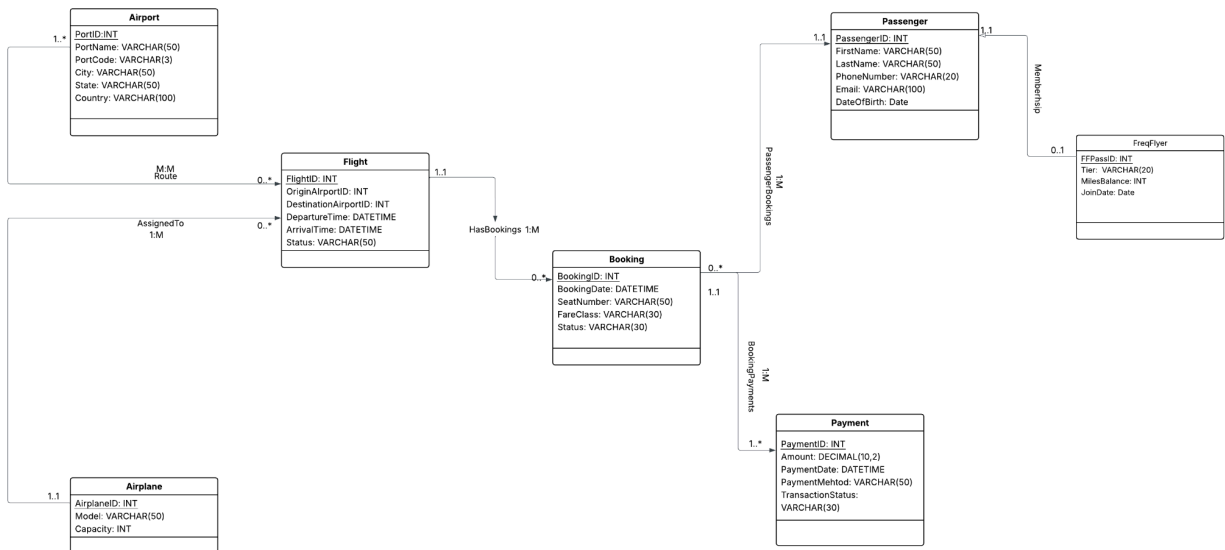
Relationship (Business) Rules:

- 1. Airport - Flight (Route)
 - a. An airport can appear in multiple routes, serving as origin or destination for many flights
 - b. A flight is associated with one origin airport and one destination airport, but airports may be in involved in many flights
- 2. Airplane - Flight (AssignedTo)
 - a. Each flight is assigned to one and only one airplane
 - b. Each airplane may be assigned to many flights over time
 - c. An airplane must participate in at least one flight assignment
- 3. Flight - Booking (HasBooking)
 - a. A booking must reference exactly one flight
 - b. A flight may have zero, one, or many bookings, depending on passenger demand
- 4. Passenger - Booking(PassengerBookings)
 - a. Each booking is made by one passenger
 - b. A passenger may have zero, one, or many bookings
- 5. Passenger - FrequentFlyer (Specialization Membership)
 - a. A FrequentFlyer account exist only if the associated Passenger exists
 - b. A passenger may have at most one FrequentFlyer membership
 - c. A FrequentFlyer inherits all Passenger information, reflecting its specialization relationship
- 6. Booking - Payment (BookingPayments)
 - a. A payment must be associated with one booking
 - b. A booking may have one or multiple payments (example: installments)
 - c. A completed booking requires at least one successful payment

ERD:



UML:



Relational Schema:

- **Airport**(PortID, PortName, PortCode, City, State, Country)
- **Airplane**(AirplaneID, model, capacity)
- **Flight**(FlightID, OriginAirportID, DestinationAirportID, AirplaneID, DepartureTime, ArrivalTime, Status)
 - FK: PortID references Airport
 - FK: AirplaneID references Airplane
- **Route**(PortID, FlightID)
 - FK: PortID references Airport
 - FK: FlightID references Flight
- **Passenger**(PassengerID, FirstName, LastName, PhoneNumber, Email, DateOfBirth)
 - FK: BookingID references Booking
- **Booking**(BookingID, FlightID, Airline, BookingDate, SeatNumber, FareClass, Status)
 - FK: FlightID references Flight
- **Payment**(PaymentID, BookingID, Amount, PaymentDate, PaymentMethod, TransactionStatus)
 - FK: BookingID references Booking
- **FreqFlyer**(FFPassID, PassengerID, Tier, MilesBalance, JoinDate)
 - FK: PassengerID references Passenger

Code:

Making the tables and inserting data:

```
DROP TABLE FreqFlyer
DROP TABLE Payment
DROP TABLE Passenger
DROP TABLE Booking
DROP TABLE Flight
DROP TABLE Airplane
DROP TABLE Airport
DROP TABLE Route
```

--Airplane Table

```
Create Table Airplane(
PlaneID INT Identity(1000,1) Primary Key,
Model varchar(50) not null,
Capacity int --this is a little detail because the largest commercial carrier can only carry 853
people :)
)
```

```
INSERT INTO Airplane (Model, Capacity)
VALUES
```

```
('Boeing 737-800', 189),
('Airbus A320', 180),
('Boeing 777-300ER', 396),
('Airbus A380-800', 853),
('Boeing 787-9 Dreamliner', 296),
('Boeing 747-400', 416),
('Airbus A350-900', 325),
('Boeing 737 MAX 9', 178),
('Airbus A321neo', 220),
('Embraer E195-E2', 146),
('Boeing 767-300ER', 269),
('Airbus A330-300', 277),
('Boeing 757-200', 200),
('Airbus A220-300', 160),
('Boeing 737-900ER', 215);
```

--Airport Table

```
Create Table Airport (
PortId Int IDENTITY(100,1) Primary Key,
PortName VarChar(50),
PortCode VarChar(3),
City VarChar(50),
State Varchar(50),
Country Varchar (100)
)
```

insert into Airport (PortName, PortCode, City, State, Country) values

```
('San Diego International Airport', 'SAN', 'San Diego', 'California', 'USA'),
('Los Angeles International Airport', 'LAX', 'Los Angeles', 'California', 'USA'),
('John F. Kennedy International Airport', 'JFK', 'New York', 'New York', 'USA'),
('Hartsfield-Jackson Atlanta International', 'ATL', 'Atlanta', 'Georgia', 'USA'),
('O'Hare International Airport', 'ORD', 'Chicago', 'Illinois', 'USA'),
('Dallas/Fort Worth International', 'DFW', 'Dallas', 'Texas', 'USA'),
('Denver International Airport', 'DEN', 'Denver', 'Colorado', 'USA'),
('San Francisco International Airport', 'SFO', 'San Francisco', 'California', 'USA'),
('Seattle-Tacoma International Airport', 'SEA', 'Seattle', 'Washington', 'USA'),
('Miami International Airport', 'MIA', 'Miami', 'Florida', 'USA'),
('Phoenix Sky Harbor International', 'PHX', 'Phoenix', 'Arizona', 'USA'),
('Boston Logan International Airport', 'BOS', 'Boston', 'Massachusetts', 'USA'),
```

('Newark Liberty International Airport', 'EWR', 'Newark', 'New Jersey', 'USA'),
('Orlando International Airport', 'MCO', 'Orlando', 'Florida', 'USA'),
('Charlotte Douglas International', 'CLT', 'Charlotte', 'North Carolina', 'USA');

--Flight Table:

```
Create table Flight (  
FlightID Int Identity(1000,1) Primary Key,  
OriginAirportID int foreign key references Airport(PortId),  
DestinationAirportID int foreign key references Airport(PortId),  
AirplaneID int foreign key references Airplane(PlaneID),  
DepartureTime DateTime,  
ArrivalTime DateTime,  
Status varchar(50) --"Scheduled", "Delayed", "Cancelled"  
);
```

```
INSERT INTO Flight (OriginAirportID, DestinationAirportID, AirplaneID, DepartureTime,  
ArrivalTime, Status)
```

VALUES

```
(100, 101, 1000, '2024-12-01 08:00:00', '2024-12-01 11:30:00', 'Scheduled'),  
(101, 102, 1001, '2024-12-02 06:30:00', '2024-12-02 09:00:00', 'Delayed'),  
(102, 104, 1002, '2024-12-02 12:00:00', '2024-12-02 15:20:00', 'Cancelled'),  
(104, 100, 1003, '2024-12-03 19:45:00', '2024-12-03 22:10:00', 'Scheduled'),  
(105, 106, 1004, '2024-12-03 14:00:00', '2024-12-03 18:30:00', 'Scheduled'),  
(107, 108, 1005, '2024-12-04 07:15:00', '2024-12-04 10:45:00', 'Scheduled'),  
(108, 109, 1006, '2024-12-04 16:30:00', '2024-12-04 21:00:00', 'Delayed'),  
(110, 111, 1007, '2024-12-05 09:00:00', '2024-12-05 12:30:00', 'Scheduled'),  
(111, 112, 1008, '2024-12-05 13:45:00', '2024-12-05 17:15:00', 'Scheduled'),  
(112, 100, 1009, '2024-12-06 11:30:00', '2024-12-06 15:00:00', 'Cancelled'),  
(100, 113, 1010, '2024-12-06 18:00:00', '2024-12-06 22:30:00', 'Scheduled'),  
(113, 101, 1011, '2024-12-07 05:45:00', '2024-12-07 09:15:00', 'Scheduled'),  
(101, 114, 1012, '2024-12-07 15:30:00', '2024-12-07 19:00:00', 'Delayed'),  
(114, 104, 1013, '2024-12-08 08:30:00', '2024-12-08 12:00:00', 'Scheduled'),  
(104, 107, 1014, '2024-12-08 19:15:00', '2024-12-08 23:45:00', 'Scheduled'),  
(107, 110, 1000, '2024-12-09 06:00:00', '2024-12-09 09:30:00', 'Scheduled'),  
(110, 100, 1001, '2024-12-09 14:45:00', '2024-12-09 18:15:00', 'Scheduled'),  
(100, 108, 1002, '2024-12-10 10:30:00', '2024-12-10 14:00:00', 'Scheduled'),  
(108, 112, 1003, '2024-12-10 20:00:00', '2024-12-11 00:30:00', 'Scheduled'),  
(112, 105, 1004, '2024-12-11 12:15:00', '2024-12-11 15:45:00', 'Scheduled');
```

--Booking Table:

```
Create TABLE Booking(  
BookingID Int Identity(10000,1) Primary Key,  
FlightID Int Foreign Key references Flight(FlightID),
```

```
BookingDate DateTime,  
SeatNumber VarChar(5),  
FareClass Varchar (30), -- "Business", "Premium Economy" "Economy"  
Status Varchar(30) -- "Checked-in","Cancelled","Confirmed"  
)
```

```
Insert INTO Booking (FlightID, BookingDate, SeatNumber, FareClass, Status)  
Values
```

```
(1000, '2024-11-15 14:30:00', '12A', 'Business', 'Confirmed'),  
(1000, '2024-11-16 09:20:00', '23C', 'Economy', 'Checked-in'),  
(1001, '2024-11-20 11:45:00', '8F', 'Premium Economy', 'Confirmed'),  
(1002, '2024-11-22 16:00:00', '45B', 'Economy', 'Cancelled'),  
(1003, '2024-11-25 10:15:00', '1A', 'Business', 'Checked-in'),  
(1004, '2024-11-26 08:30:00', '15D', 'Economy', 'Confirmed'),  
(1005, '2024-11-27 13:45:00', '22B', 'Business', 'Confirmed'),  
(1006, '2024-11-28 16:20:00', '31A', 'Economy', 'Checked-in'),  
(1007, '2024-11-29 10:00:00', '5C', 'Premium Economy', 'Confirmed'),  
(1008, '2024-11-30 14:15:00', '18E', 'Economy', 'Confirmed'),  
(1009, '2024-12-01 07:45:00', '9B', 'Business', 'Cancelled'),  
(1010, '2024-12-02 11:30:00', '27F', 'Economy', 'Confirmed'),  
(1011, '2024-12-03 15:00:00', '14A', 'Premium Economy', 'Checked-in'),  
(1012, '2024-12-04 09:15:00', '33D', 'Economy', 'Confirmed'),  
(1013, '2024-12-05 12:45:00', '7B', 'Business', 'Confirmed'),  
(1014, '2024-12-06 18:30:00', '20C', 'Economy', 'Checked-in'),  
(1015, '2024-12-07 08:00:00', '11E', 'Premium Economy', 'Confirmed'),  
(1016, '2024-12-08 13:20:00', '25A', 'Economy', 'Confirmed'),  
(1017, '2024-12-09 16:45:00', '6F', 'Business', 'Confirmed'),  
(1018, '2024-12-10 10:30:00', '19D', 'Economy', 'Checked-in');
```

--Passenger Table:

```
CREATE TABLE Passenger (  
    PassengerID INT IDENTITY(10000,1) PRIMARY KEY,  
    BookingID INT NOT NULL Foreign Key References Booking(BookingID),  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    Phone VARCHAR(20),  
    Email VARCHAR(100),  
    DateOfBirth DATE,  
);
```

```
INSERT INTO Passenger (BookingID, FirstName, LastName, Phone, Email, DateOfBirth)  
VALUES  
(10000, 'John', 'Doe', '555-1234', 'john@email.com', '1990-05-15'),
```

```
(10001, 'Sarah', 'Johnson', '555-2345', 'sarah.j@email.com', '1985-08-22'),
(10002, 'Michael', 'Chen', '555-3456', 'mchen@email.com', '1992-03-10'),
(10003, 'Emily', 'Williams', '555-4567', 'ewilliams@email.com', '1988-11-30'),
(10004, 'David', 'Martinez', '555-5678', 'dmartinez@email.com', '1995-07-18'),
(10005, 'Jennifer', 'Garcia', '555-6789', 'jgarcia@email.com', '1987-02-14'),
(10006, 'Robert', 'Anderson', '555-7890', 'randerson@email.com', '1993-09-25'),
(10007, 'Lisa', 'Taylor', '555-8901', 'ltaylor@email.com', '1991-06-08'),
(10008, 'James', 'Thomas', '555-9012', 'jthomas@email.com', '1989-12-03'),
(10009, 'Maria', 'Rodriguez', '555-0123', 'mrodriguez@email.com', '1994-04-17'),
(10010, 'William', 'Brown', '555-1357', 'wbrown@email.com', '1986-11-22'),
(10011, 'Patricia', 'Wilson', '555-2468', 'pwilson@email.com', '1992-07-30'),
(10012, 'Christopher', 'Lee', '555-3579', 'clee@email.com', '1990-01-11'),
(10013, 'Linda', 'Moore', '555-4680', 'lmoore@email.com', '1988-08-19'),
(10014, 'Daniel', 'Jackson', '555-5791', 'djackson@email.com', '1993-03-27'),
(10015, 'Barbara', 'White', '555-6802', 'bwhite@email.com', '1987-10-05'),
(10016, 'Matthew', 'Harris', '555-7913', 'mharris@email.com', '1991-05-14'),
(10017, 'Susan', 'Martin', '555-8024', 'smartin@email.com', '1989-12-23'),
(10018, 'Anthony', 'Thompson', '555-9135', 'athompson@email.com', '1994-06-09'),
(10019, 'Jessica', 'Clark', '555-0246', 'jclark@email.com', '1992-02-28');
```

--Frequent Flyer

```
Create Table FreqFlyer(
FFPassID INT PRIMARY KEY,
Tier Varchar(20),
MilesBalance Int,
JoinDate Date,
FOREIGN KEY (FFPassID) REFERENCES Passenger(PassengerID)
)
```

INSERT INTO FreqFlyer (FFPassID, Tier, MilesBalance, JoinDate)

VALUES

```
(10000, 'Gold', 75000, '2020-03-15'),
(10001, 'Platinum', 125000, '2019-06-22'),
(10002, 'Silver', 45000, '2021-09-10'),
(10004, 'Gold', 82000, '2020-11-18'),
(10006, 'Bronze', 15000, '2023-02-14'),
(10007, 'Silver', 52000, '2021-05-25'),
(10009, 'Platinum', 150000, '2018-12-03'),
(10011, 'Gold', 68000, '2020-07-30'),
(10013, 'Bronze', 22000, '2022-08-19'),
(10015, 'Silver', 48000, '2021-10-05');
```

--Payment Table:


```

Create Table Payment(
PaymentID Int Identity(1000,1) Primary Key,
BookingID int foreign key references Booking(BookingID),
Amount Decimal (10,2),
PaymentDate DateTime,
PaymentMethod Varchar(50), --Visa, Discover, MasterCard
TransactionStatus VarChar(30) -- I don't know what this would be yet
)

```

```

INSERT INTO Payment (BookingID, Amount, PaymentDate, PaymentMethod,
TransactionStatus)
VALUES

```

```

(10000, 850.00, '2024-11-15 14:35:00', 'Visa', 'Completed'),
(10001, 320.50, '2024-11-16 09:25:00', 'MasterCard', 'Completed'),
(10002, 475.00, '2024-11-20 11:50:00', 'Discover', 'Completed'),
(10003, 280.00, '2024-11-22 16:05:00', 'Visa', 'Refunded'),
(10004, 1200.00, '2024-11-25 10:20:00', 'MasterCard', 'Completed'),
(10005, 395.75, '2024-11-26 08:35:00', 'Visa', 'Completed'),
(10006, 920.00, '2024-11-27 13:50:00', 'American Express', 'Completed'),
(10007, 410.50, '2024-11-28 16:25:00', 'MasterCard', 'Completed'),
(10008, 565.00, '2024-11-29 10:05:00', 'Discover', 'Completed'),
(10009, 340.25, '2024-11-30 14:20:00', 'Visa', 'Completed'),
(10010, 785.00, '2024-12-01 07:50:00', 'MasterCard', 'Pending'),
(10011, 495.50, '2024-12-02 11:35:00', 'Visa', 'Completed'),
(10012, 625.00, '2024-12-03 15:05:00', 'American Express', 'Completed'),
(10013, 380.75, '2024-12-04 09:20:00', 'Discover', 'Completed'),
(10014, 1050.00, '2024-12-05 12:50:00', 'MasterCard', 'Completed'),
(10015, 440.00, '2024-12-06 18:35:00', 'Visa', 'Completed'),
(10016, 515.25, '2024-12-07 08:05:00', 'American Express', 'Completed'),
(10017, 670.00, '2024-12-08 13:25:00', 'MasterCard', 'Completed'),
(10018, 890.50, '2024-12-09 16:50:00', 'Visa', 'Completed'),
(10019, 355.00, '2024-12-10 10:35:00', 'Discover', 'Completed');

```

--Route table:

```

CREATE TABLE Route (
FlightID INT NOT NULL,
OriginPortID INT NOT NULL,
DestPortID INT NOT NULL,
PRIMARY KEY (FlightID, OriginPortID, DestPortID),
FOREIGN KEY (FlightID) REFERENCES Flight(FlightID),
FOREIGN KEY (OriginPortID) REFERENCES Airport(PortID),
FOREIGN KEY (DestPortID) REFERENCES Airport(PortID)
);

```

```
-- Insert Origin Airport into Route
INSERT INTO Route (FlightID, OriginPortID, DestPortID)
SELECT FlightID, OriginAirportID, DestinationAirportID
FROM Flight;
```

Queries and selections:

```
SELECT
    FareClass,
    COUNT(BookingID) as TotalBookings
FROM Booking
WHERE Status != 'Cancelled'
GROUP BY FareClass
ORDER BY TotalBookings DESC
```

FareClass	TotalBookings
Economy	9
Business	5
Premium Economy	4

```
SELECT
    PortName,
    PortCode,
    City,
    COUNT(F.FlightID) as DepartingFlights
FROM Airport as A
JOIN Flight F ON A.PortId = F.OriginAirportID
GROUP BY A.PortName, A.PortCode, A.City
ORDER BY DepartingFlights DESC
```

PortName	PortCode	City	DepartingFlights
San Diego International Airport	SAN	San Diego	3
San Francisco International Airport	SFO	San Francisco	2
Seattle-Tacoma International Airport	SEA	Seattle	2
Phoenix Sky Harbor International	PHX	Phoenix	2
Los Angeles International Airport	LAX	Los Angeles	2
Newark Liberty International Airport	EWB	Newark	2
O'Hare International Airport	ORD	Chicago	2
Orlando International Airport	MCO	Orlando	1

```

SELECT
    YEAR(B.BookingDate) as Year, MONTH(B.BookingDate) as Month, COUNT(B.BookingID) as TotalBookings, AVG(P.Amount) as AvgTicketPrice, MIN(P.Amount) as LowestPrice, MAX(P.Amount) as HighestPrice
FROM Booking as B
JOIN Payment as P
ON B.BookingID = P.BookingID
WHERE P.TransactionStatus = 'Completed'
GROUP BY YEAR(B.BookingDate), MONTH(B.BookingDate), DATENAME(MONTH, B.BookingDate)
ORDER BY Year, Month

```

Year	Month	TotalBookings	AvgTicketPrice	LowestPrice	HighestPrice
2024	11	9	608.555555	320.50	1200.00
2024	12	9	602.444444	355.00	1050.00

```

SELECT
    F.FlightID,
    Origin.PortCode as OriginCode,
    Origin.City as OriginCity,
    Dest.PortCode as DestinationCode,
    Dest.City as DestinationCity,
    F.DepartureTime
FROM Flight F
JOIN Airport Origin ON F.OriginAirportID = Origin.PortId
JOIN Airport Dest ON F.DestinationAirportID = Dest.PortId
LEFT JOIN Booking B ON F.FlightID = B.FlightID
WHERE F.Status = 'Cancelled'
ORDER BY F.DepartureTime

```

FlightID	OriginCode	OriginCity	DestinationCode	DestinationCity	DepartureTime
1002	JFK	New York	ORD	Chicago	12/2/2024 12:00:00 PM
1009	EWR	Newark	SAN	San Diego	12/6/2024 11:30:00 AM

Resources Used:

Throughout this project, we utilized a variety of tools and resources to support our design, research, and collaboration efforts. Lucidchart was used to create UML and ERD diagrams from start to finish, allowing us to adapt to increasing database complexity. Official FAA and airline documentation provided accurate and reliable information to properly contextualize and structure the data within our database. We used SQL Server Virtual Machine via MyEducator to build, edit, query, and manage our database tables, including creating, linking, updating, and viewing records. Google Drive and Workspace enabled collaborative brainstorming, document sharing, and consolidation of our work in a single platform, while Zoom facilitated virtual communication and coordination of project milestones.

Pros/Cons

The Database system has several meaningful attributes such as the reliability to capture and retain critical operational data, the compatibility to intertwine with existing airline databases while maintaining government security/accuracy, and the fundamentals to expand into a large scale database, if involved with larger entities. On the contrary, there are some potential limitations to our database such as the inability to accurately track pilot/gate data, the possibility of user identity issues as the autonumber system could fail, and lastly the non-invasive user information side for special circumstances.

Conclusion

In conclusion, the Domestic Airlines database project enabled us to build a functional, realistic model that reflects how an airline manages its daily operations. By creating entities such as Airport, Airplane, Flight, Passenger, Booking, Payment, and FrequentFlyer, we designed a system that captures the essential data required to operate a domestic airline efficiently. The relationships we built, such as assigning airplanes to flights, linking passengers to bookings, and connecting payments to reservations, gave us a clear structure for how information flows throughout the airline.